ENGINEERING REQUIREMENTS

Google Photos Integration

Implementation requirements, API contracts, security, and acceptance tests

PROJECT 2	REVISION 1.0	11 NOV 2026

Decision: integrate Google Photos Picker for user-initiated selection. Import selected bytes into the existing private Google Cloud Storage bucket for editing. Do not use Google Drive, scrape Photos, or assume Firebase Google sign-in grants Photos API access.

1. Goal and boundaries

Deliver an end-to-end source-image flow in which a signed-in Pinboard Studio user can authorize Google Photos, search and select a personal photo in Google’s Picker, import one selected image into a user-owned creation, and continue into the AI editing workflow. The API and UI must enforce Google Photos policy, Firebase ownership, secure OAuth handling, image validation, and cleanup.

In scope

- Google Photos Picker API integration using user OAuth and the minimum selection scope.
- OAuth connection UX distinct from Firebase login; state/CSRF protection and secure server-side token handling.
- Picker session create, polling, list selected items, import selected image bytes, session cleanup, and cancellation/timeout paths.
- Source-image validation, private Cloud Storage import, Firestore reference metadata, preview, and deletion.
- In-context notice and affirmative consent for importing Photos content and for sending it to the AI provider.

Out of scope for this integration

- Pinterest OAuth, Pinterest board/pin APIs, and Pinterest publishing.
- Google Drive.
- Unrestricted search or enumeration of all Google Photos library items.
- Editing/overwriting the user's original Photos item. Optional export creates a new app-created media item and needs separate scope and consent.
- A general-purpose photo gallery, face clustering, advertising, resale, or training a general AI model on Photos data.

2. Prerequisites and external setup

Prerequisite	Required action / evidence
Google Cloud project	Use `project-2-pinboard-2026` unless the project owner chooses otherwise. Enable the Google Photos Picker API (not the similarly named Google Picker API). Confirm quota and API availability.
OAuth consent and Web client	Configure consent-screen branding, support contact, authorized origins for Firebase Hosting and localhost, and backend callback URI. Keep client secret server-side. Add development test users.
Photos OAuth scope	Request only `https://www.googleapis.com/auth/photospicker.mediaitems.readonly` for Picker session creation/status/deletion and selected-media listing. Recheck current verification classification before release.
User experience and policy	Publish a privacy policy and data deletion help page. Add visible just-in-time disclosures for selection/import and separate disclosure/affirmative consent before transfer to Gemini.
Runtime secret and IAM	Store OAuth client secret using Secret Manager or equivalent. Grant Cloud Run only required secret access. Google Photos APIs do not support service accounts for user Photos data; use delegated user OAuth tokens.
Current API foundation	Reuse Firebase Auth, the existing owner-scoped creation API, private media bucket, Storage IAM, and short-lived media URL endpoints. Do not expose service credentials in Vite variables.

3. End-to-end user flow

1. Authenticate to Pinboard. Require a valid Firebase ID token. The API uses the verified UID for all Firestore and object paths.

2. Explain and authorize. In the context of choosing a source image, explain the Photos Picker scope, that the user selects the item, that the app imports a working copy to private Cloud Storage, and how the user disconnects/deletes it. Require affirmative action. Start a separate Google OAuth grant; do not treat Firebase's Google login token as a Photos API grant.

3. Create Picker session. Backend calls `sessions.create` using the user's OAuth token and associates the Google session ID with the authenticated UID and target creation. Return only the pickerUri and an opaque app session ID to the UI.

4. Open Picker. UI opens pickerUri in a new tab/window; do not iframe. Explain that Google Photos opens for search/selection. Support user cancel, blocked popup, timeout, and retry.

5. Poll. Backend/UI polls session state using the returned `pollingConfig.pollInterval` and stops on `mediaItemsSet`, timeout, cancellation, or error. Do not hard-code aggressive polling.

6. List selected items. After selection is complete, backend calls Picker API list and returns only the selected-item data needed to display the selection. Do not query arbitrary library content.

7. Choose one in Pinboard. For MVP, if multiple items were selected in Photos, display a small selection grid and require the user to choose one source item. Enforce file-count limits before importing.

8. Import bytes promptly. Backend uses the user's authorized Photos token and selected item's temporary baseUrl to fetch bytes over HTTPS. Do not return or persist the baseUrl. Validate response status, content length, declared MIME type, file signature, successful safe decode, and dimensions. Copy approved bytes to a generated UID/creation-scoped staging key in the private GCS bucket.

9. Finalize. Move/promote validated object to `users/{uid}/creations/{creationId}/sourceImage/{objectId}`. Update Firestore with object path and normalized metadata. Delete staged bytes and Picker session after retrieval/import according to Google's documented session lifecycle.

10. Preview and edit. Obtain an ownership-checked five-minute signed preview URL. Before Gemini processing, show what is sent and require affirmative consent. Send the image only for this user's visible requested edit; do not use Photos-derived content to train or improve a general model.

11. Cleanup. On failure/cancel, delete staged files and temporary records. On creation deletion or user deletion request, remove source/output objects and associated app data/tokens. Revoke OAuth grant on disconnect where supported.

4. OAuth and credential requirements

- Model Firebase Auth UID and Google OAuth subject as separate identities; link only after the currently signed-in Pinboard user explicitly completes the OAuth flow.
- Use a maintained Google OAuth client library. Validate unpredictable `state`, exact redirect URI, callback state, and token audience/client. Use PKCE where supported by the selected web OAuth flow.
- Keep client secret and access/refresh tokens out of the browser bundle, URLs, logs, analytics, and API responses. Never log `Authorization` headers, tokens, picker session IDs if they can be used as credentials, or base URLs.
- Choose the smallest viable token lifetime/retention. If refresh tokens are kept for reconnect-free usage, encrypt them at application level using Cloud KMS-backed envelope encryption, restrict decrypt IAM, record requested scopes and expiry, and support revocation/deletion. Do not store plaintext tokens in Firestore.
- Use a separate OAuth connection status and disconnect control. Disconnect must stop future Photos calls, revoke/delete stored credentials, and clearly distinguish whether existing app copies are retained or deleted.
- Google Photos APIs do not support service accounts. Cloud Run workload identity is still appropriate for GCS/Firestore, but it does not replace user OAuth for Photos.

5. Proposed API contract

Method / endpoint	Purpose / expected behavior
POST `/api/integrations/google-photos/oauth/start`	Firebase-authenticated. Start OAuth with state/PKCE; return redirect URL or redirect response. Request Picker scope only.
GET `/api/integrations/google-photos/oauth/callback`	Validate state/code, exchange server-side, persist protected token material, associate Google account to verified Firebase UID, and redirect to UI success/error route.
GET `/api/integrations/google-photos/status`	Return connected boolean and non-sensitive account display metadata only; never return tokens.
DELETE `/api/integrations/google-photos`	Revoke and remove stored token material/connection metadata. If requested, also delete Photos-imported app media according to explicit UX.
POST `/api/creations/:creationId/google-photos/picker-sessions`	Require Firebase user and owned active creation. Create Picker session using linked Google OAuth. Return opaque app session ID, pickerUri, and expiration/status data.
GET `/api/creations/:creationId/google-photos/picker-sessions/:sessionId`	Verify owner/session binding, poll Google session, and return state plus selected items after `mediaItemsSet=true`. Respect recommended polling configuration.
POST `/api/creations/:creationId/google-photos/picker-sessions/:sessionId/import`	Body contains selected Picker item ID, not arbitrary URL. Verify it belongs to that session, fetch bytes server-side, validate, stage/promote to GCS, and attach Firestore media reference.
DELETE `/api/creations/:creationId/google-photos/picker-sessions/:sessionId`	Owner-only cancellation/cleanup; delete Google session when supported and remove local state/staged objects.

API contract rules

- Every application route except OAuth callback/health is authenticated with Firebase ID token. The callback must bind its state to a single-use server-side OAuth transaction and Firebase UID.

- Do not accept arbitrary external image URLs from the browser for the Photos import endpoint; this prevents SSRF and ensures the selected item belongs to the Picker session.
- Return stable API errors for unlinked account, consent denied, expired picker session/base URL, unavailable item, quota/rate limit, unsupported media, oversized image, and import failure.
- Use idempotency or a session/import state machine so retrying import cannot create duplicate creation media or leave two final objects.

6. Image validation and storage requirements

Validation / control	Requirement
Allowed formats	MVP: JPEG, PNG, WebP. Confirm GIF/HEIC policy separately; reject anything not explicitly decoded and supported.
Byte limit	Use the existing image maximum of 10 MiB unless product/AI limits justify changing it. Enforce both declared and actual bytes.
File signature	Inspect magic bytes and decode with a maintained image library (for example Sharp/libvips); do not trust filename or client MIME alone.
Dimensions	Set maximum width, height, and total pixels; reject malformed and decompression-bomb images. Record width/height from decoded content.
Metadata safety	Strip EXIF/location metadata from the working copy unless there is a documented user-facing reason to preserve it.
Object key	`users/{uid}/creations/{creationId}/sourceImage/{serverGeneratedId}`. Never use user filenames as path components.
Object ACL/access	Private bucket, uniform bucket-level access, public access prevention, bucket-scoped runtime IAM, no public ACL. Preview via short-lived signed URL after owner check.
Orphan cleanup	Clean staging on success/failure/cancel; one-day bucket lifecycle as backstop; Firestore TTL for temporary sessions; creation deletion removes matching staged and final objects.

7. Firestore data model changes

Keep the existing user and creation layout. Add only fields required by this integration; never store the selected image bytes or expiring Photos base URL.

Document / proposed fields	Notes
`users/{uid}`: `googlePhotos.connected`, `googlePhotos.googleSubject` (optional), `googlePhotos.connectedAt`, `googlePhotos.scopes`	Non-secret connection metadata only. Avoid copying unnecessary Google profile data.
`users/{uid}/oauthConnections/googlePhotos`: encrypted token payload, `expiresAt`, `scopes`, `updatedAt`	Only if persistent OAuth is required. Token ciphertext only; restrict API/IAM access; delete on disconnect. If product can reauthorize per use, avoid long-lived refresh-token storage.
`users/{uid}/creations/{creationId}.media.sourceImage`: `objectPath`, `contentType`, `sizeBytes`, `width`, `height`, `updatedAt`	Reuse current media reference shape and add validated dimensions.
`users/{uid}/creations/{creationId}.sourceProvider`: `google_photos` or `direct_upload`	Records provenance at a coarse level. Do not use as a durable photo URL.
`users/{uid}/photoPickerSessions/{appSessionId}`: `providerSessionId`, `creationId`, `status`, `expiresAt`, `createdAt`	Temporary owner-bound state; no raw OAuth token or base URL. Enable Firestore TTL on expiresAt.

8. UI requirements

- Replace simulated Pinterest connect/board/pin UI with a clearly labeled Google Photos selection action. Maintain a distinct local-file upload fallback if the product wants it.
- Explain the Photos access and AI processing before authorization/transfer; present privacy-policy and deletion-help links.
- Open Google’s picker in a new tab/window, not iframe. Explain what to do if a popup is blocked; preserve the app state during return.
- Show connecting, waiting for selection, polling, selected items, import progress, validation errors, retry, cancellation, and success states accessibly.
- For MVP, allow one final image per creation and make the user choose one if several were picked. Provide attribution/source indication only if policy/product requirements permit; do not claim Pinterest functionality.
- After import, show a preview from the app's signed URL and provide replace/remove controls. Never display an expired Google base URL as the source.
- Show separate status for Firebase signed-in state and Google Photos connected state, including reconnect/disconnect.

9. Failure, privacy, and operations

- Use server-side OAuth state and safe redirect allowlists; prevent account-linking CSRF and cross-user session reuse.
- Limit picker polling frequency using Google's `pollingConfig`; handle 401/403/429/5xx with documented retries/backoff and clear user messages.
- Use bounded retry with exponential backoff for transient Google API errors; do not repeatedly retry consent denial, permission denial, expired session, or invalid media.
- Apply rate limits to session creation/import routes and cap pending sessions per user to control abuse and quota use.
- Log request IDs, Firebase UID (or a suitable pseudonymous identifier), session state transitions, latency, and error class; never log tokens, base URLs, or photo bytes.
- Define retention for source/output objects and deletion UX. Honor deletion requests for Firestore data, GCS objects, OAuth credentials, and supported Google sessions.
- Google Photos data is used only for the user-facing purpose. Do not sell/share for advertising or use it to train/improve a general model. Any Gemini processing must be explicitly disclosed and consented to; verify provider terms and Google's Limited Use policy before enabling it.
- Review Google OAuth verification, privacy policy, quota, and API terms before public production use. Re-check requirements because API policies can change.

10. Test plan and acceptance criteria

Test area	Pass condition
OAuth	Valid consent links the Google account to the Firebase UID; bad/replayed state and wrong callback are rejected; tokens never appear in UI/API response/log.
Picker flow	Create session, open pickerUri externally, poll at recommended cadence, detect selection complete, list only selected items, and clean session.
Ownership	User A cannot poll/import/delete user B's app session, creation, or media. Arbitrary baseUrl/item IDs are not accepted as import authority.
Expiration	Expired OAuth token/session/base URL gives a recoverable reauthorize/reselect path; no stale baseUrl is persisted or reused.
Validation	Reject wrong signatures, spoofed MIME/extension, unsupported type, oversize bytes, excessive dimensions/pixels, corrupt image, and excessive selection count.
Import/storage	Successful import produces one private object at expected UID/creation path and a matching Firestore metadata reference; no public object URL.
Cleanup	Cancel, timeout, import error, creation deletion, and disconnect remove applicable session/staging/token state; no usable orphan object remains.
AI consent	No Photos-derived bytes reach Gemini before clear affirmative user consent. Test consent denial leaves image selection intact without processing.
Browser	Production and localhost OAuth origins and bucket/API CORS work; blocked popup, cancellation, and return-to-tab behavior work.
Operations	CI unit/integration checks pass; live smoke test uses a test account and a user-selected test image; logs contain no secrets or image data.

11. Work breakdown and dependencies

Order	Work item	Depends on
1	Confirm Google Photos policy/AI consent design, choose OAuth verification audience, privacy/retention statements.	Project owner/professor decision; before real Photos data is processed.
2	Enable Picker API, configure OAuth Web client, consent screen, origins/redirects, test users, secrets.	1
3	Backend OAuth callback/state, secure token storage/revocation, Google Photos client.	2
4	Picker session API/state machine, polling, selected-item list, cleanup.	3
5	Image import, decode validation, safe object keys, GCS/Firestore metadata.	4; existing media lifecycle.
6	React picker launch, selection review, progress, errors, consent, preview.	4-5; UI repo `bhushannakhate/Web-System`.
7	Gemini transfer behind explicit consent; editing result storage/review.	1 and 5; existing edit-generation work.
8	Security tests, live tester E2E, OAuth verification readiness, deployment and documentation.	3-7

12. Release gates / decisions needed

- Approve Google Photos as the replacement selection provider and remove Pinterest API dependencies from the active release scope.
- Choose whether users must reauthorize per session or whether encrypted refresh-token storage is necessary for a persistent connection.
- Approve the exact user-facing disclosure and retention/deletion period for imported originals and generated images.
- Confirm Gemini provider terms/data-handling behavior are compatible with Google Photos data use and the project's consent flow before enabling image transfer.
- Confirm if saving generated images back into Google Photos is required for the project grade. If yes, add a separate feature using the Library API app-created-media scope and a second in-context consent.

Official references

The API behavior and policy requirements below were checked against Google documentation on 11 November 2026. Re-check the current documentation before implementation and release.

Picker API: Get started — OAuth, create a session, open pickerUri, poll, list, retrieve.

Picker API: Sessions — Session lifecycle, pollingConfig, cleanup, and pickerUri behavior.

Picker API: Retrieve media items — Selected-item metadata and time-limited base URLs.

Google Photos API authorization scopes — Picker scope and app-created-data Library scopes.

Configure Google Photos APIs — Enable the correct API, configure OAuth client, web origins, and redirect URIs.

Photos API user-data policy — Appropriate use, disclosure, consent, data handling, and Limited Use rules.

Google Photos API UX guidelines — In-context disclosure, user control, and consent experience.

Google Photos API quotas — Current quotas and usage limits.